

dispatch_layout算子主要逻辑：计算当前rank的token选择的专家分布，和token在专家上的小偏移

token_num=5
expert_num=16
rank_num=4
core_num=3

topk_idx, 每个token选中专家id

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token0	3	11	7	15	0	9	2	14
token1	9	2	15	0	12	5	13	7
token2	12	5	0	8	15	6	10	1
token3	15	0	4	13	9	2	11	7
token4	6	14	1	10	0	15	8	3

分核统计token选中expert次数

num_tokens_per_expert on single core

	e0	e1	e2	e3	e4	e5	e6	e7	e8	e9	e10	e11	e12	e12	e14	e15
aiv0	2	0	2	1	0	1	0	2	0	2	0	1	1	1	1	2
aiv1	2	1	1	0	1	1	1	1	1	1	1	1	1	1	0	2
aiv2	1	1	0	1	0	0	1	0	1	0	1	0	0	0	1	1

所有核往GM累加

输出1: num_tokens_per_expert on GM

5	2	3	2	1	2	2	3	2	3	2	2	2	2	2	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

按核粒度，求 num_tokens_per_expert 前缀和

	e0	e1	e2	e3	e4	e5	e6	e7	e8	e9	e10	e11	e12	e12	e14	e15
aiv0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
aiv1	2	0	2	1	0	1	0	2	0	2	0	1	1	1	1	2
aiv2	4	1	3	1	1	2	1	3	1	3	1	2	2	2	1	4

输出3: send_token_idx_info on GM, 即小偏移

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token0	0	0	0	0	0	0	0	0
token1	1	1	1	1	0	0	0	1
token2	1	1	2	0	2	0	0	0
token3	3	3	0	1	2	2	1	2
token4	1	1	1	1	4	4	1	1

分核统计token在rank出现次数

num_tokens_per_rank on single core

	rank0	rank1	rank2	rank3
aiv0	2	2	2	2
aiv1	2	2	2	2
aiv2	1	1	1	1

所有核往GM累加

输出4: num_tokens_per_rank on GM

5	4	5	5
---	---	---	---

dispatch_notify算子主要逻辑：汇总所有rank上选择专家的分布，计算token发往其他rank专家时的大偏移

rank1/2/3/4 token_num=5/10/4/6

expert_num=16

rank_num=4

core_num=2

token_per_expert: token选中专家的分布情况

rank0 ptr0	5	2	3	2	1	2	2	3	2	3	2	2	2	2	2	5
rank1 ptr1	9	12	3	15	0	8	11	7	5	6	14	10	4	13	5	2
rank2 ptr2	7	4	10	2	8	1	9	3	6	0	5	4	2	8	1	6
rank3 ptr3	3	1	4	0	5	2	0	3	1	2	4	0	5	1	0	1

地址交换
全卡同步，
获取全部地址
write/wait flag

1

rank0	ptr0	ptr1	ptr2	ptr3
rank1	ptr0	ptr1	ptr2	ptr3
rank2	ptr0	ptr1	ptr2	ptr3
rank3	ptr0	ptr1	ptr2	ptr3

每个rank根据指针依次
拷贝到本地GM

2

输出: rank0/1/2/3: recv_data info

5	2	3	2	1	2	2	3	2	3	2	2	2	2	2	2	5
9	12	3	15	0	8	11	7	5	6	14	10	4	13	5	2	
7	4	10	2	8	1	9	3	6	0	5	4	2	8	1	6	
3	1	4	0	5	2	0	3	1	2	4	0	5	1	0	1	

提取当前rank接收
token数量

13

等待拷贝完成，全卡同步
write/wait stat

3

2	5	6	1	3	6	0	2	2	14	5	4	2	10	4	0
---	---	---	---	---	---	---	---	---	----	---	---	---	----	---	---

14

汇总得到当前rank接收总数

输出: totalCntGt

66

15

计算当前rank每个expert接收总数

输出: recvTokenPerExpert

14	11	25	16
----	----	----	----

分核处理。以aiv1视角，负责rank2/3，则依次统计expert8/9/10/11、expert12/13/14/15号专家

当遍历expert8那一列，expert8会接收2+5+6+1个token，则源rank往这个rank的写offset=0/2/7/13

当遍历expert9那一列，expert9会接收3+6+0+2个token，则源rank往这个rank的写offset=14/17/23/23，依次类推

4

输出: recvCntGt，即putOffset，卡上每个专家接收其他rank数据的大偏移

aiv0	rank0	e0 recv rank0~4				e1 recv rank0~4				e2 recv rank0~4				e3 recv rank0~4			
		0	5	14	21	24	26	38	42	43	46	49	59	63	65	80	82
	rank1	e4 recv rank0~4				e5 recv rank0~4				e6 recv rank0~4				e7 recv rank0~4			
		0	1	1	9	14	16	24	25	27	29	40	49	49	52	59	62

aiv1	rank2	e8 recv rank0~4				e9 recv rank0~4				e10 recv rank0~4				e11 recv rank0~4			
		0	2	7	13	14	17	23	23	25	27	41	46	50	52	62	66
	rank3	e12 recv rank0~4				e13 recv rank0~4				e14 recv rank0~4				e15 recv rank0~4			
		0	2	6	8	13	15	28	36	37	39	44	45	45	50	52	58

5

rank接收token数求和，除以topk=8

6

累加得到全局中token数

7

除以num_ranks得到平均负载

8

与bsPerRank对比生成容量表
超过平均值的，多处理20%

bsPerRank[4]
5 10 4 6

totalTokens

25

avgTokens

6

processCapacity[4]

6	6*1.2	6	6
---	-------	---	---

balanceMatrixLt[4, 4*2]

	rank0-start	rank0-end	rank1-start	rank1-end	rank2-start	rank2-end	rank3-start	rank3-end
rank0	0	5	-1	-1	-1	-1	-1	-1
rank1	-1	-1	0	7	-1	-1	-1	-1
rank2	-1	-1	-1	-1	0	4	-1	-1
rank3	-1	-1	-1	-1	-1	-1	0	6

先更新自己rank的负载，
最前面的token自己处理（绿色部分）

容量表减去已处理token，
即还能处理的token数

10

processCapacity[4]

1	0	2	0
---	---	---	---

结合remainTokens和processCapacity
将rank上待处理的token按容量分摊给容量大于0的rank（红色部分）

12

token总数减去已处理，
即待处理token数

remainTokens[4]

0	3	0	0
---	---	---	---

输出: balanceMatrixLt[4, 4*2]，任务均衡表

	rank0-start	rank0-end	rank1-start	rank1-end	rank2-start	rank2-end	rank3-start	rank3-end
rank0	0	5	7	8	-1	-1	-1	-1
rank1	-1	-1	0	7	-1	-1	-1	-1
rank2	-1	-1	8	10	0	4	-1	-1
rank3	-1	-1	-1	-1	-1	-1	0	6

列的视角：当前rank的token分摊到其他rank的情况

行的视角：当前rank的总负载，需要搬运数据所属的rank和范围

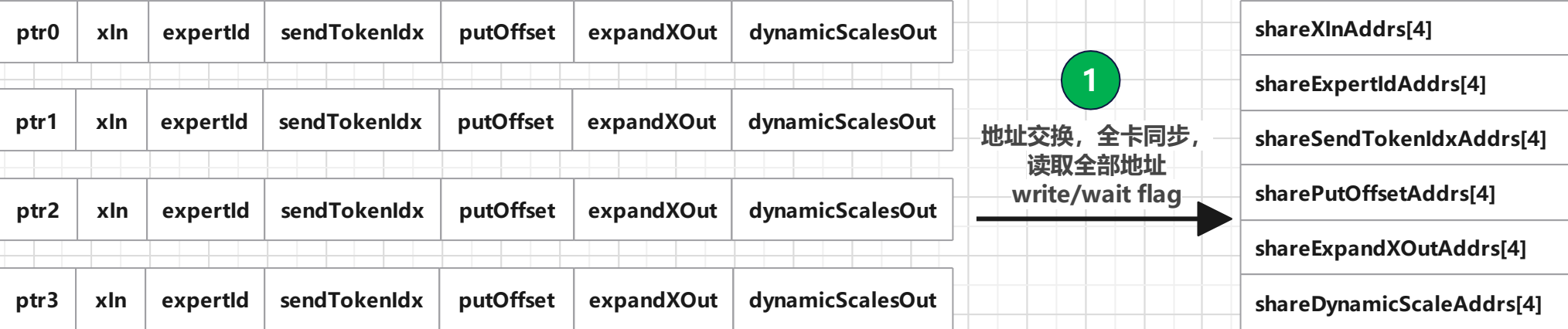
dispatch_normal算子主要逻辑：将token发送到目的rank的指定偏移

rank1/2/3/4 token_num=5/10/4/6

expert_num=16

rank_num=4

core_num=10



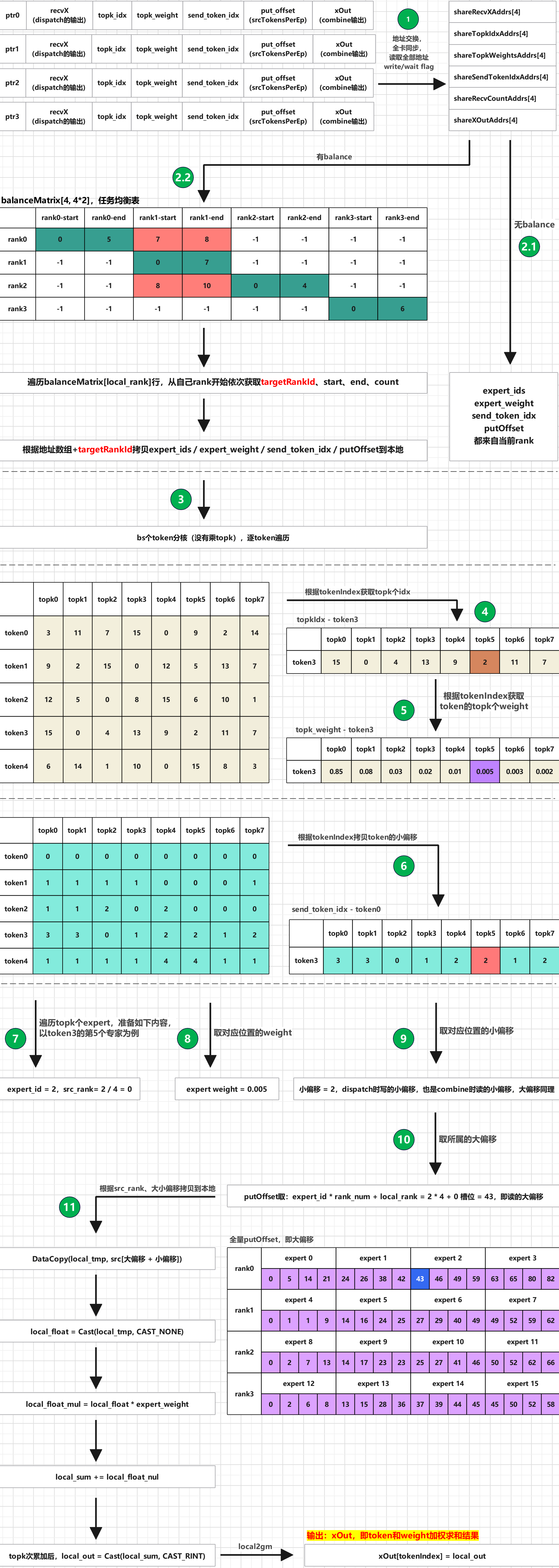
combine_normal算子主要逻辑：将发出去的token，重新收集回来

rank1/2/3/4 token_num=5/10/4/6

expert_num=16

rank_num=4

core_num=10



1

地址交换，
全卡同步，
读取全部地址
write/wait flag

shareRecvXAddr[4]

shareTopkIdxAddr[4]

shareTopkWeightsAddr[4]

shareSendTokenIdxAddr[4]

shareRecvCountAddr[4]

shareXOutAddr[4]

2.2

有balance

balanceMatrix[4, 4*2], 任务均衡表

	rank0-start	rank0-end	rank1-start	rank1-end	rank2-start	rank2-end	rank3-start	rank3-end
rank0	0	5	7	8	-1	-1	-1	-1
rank1	-1	-1	0	7	-1	-1	-1	-1
rank2	-1	-1	8	10	0	4	-1	-1
rank3	-1	-1	-1	-1	-1	-1	0	6

2.1

无balance

遍历balanceMatrix[local_rank]行，从自己rank开始依次获取targetRankId、start、end、count

根据地址数组+targetRankId拷贝expert_ids / expert_weight / send_token_idx / putOffset到本地

expert_ids
expert_weight
send_token_idx
putOffset
都来自当前rank

3

bs个token分核（没有乘topk），逐token遍历

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token0	3	11	7	15	0	9	2	14
token1	9	2	15	0	12	5	13	7
token2	12	5	0	8	15	6	10	1
token3	15	0	4	13	9	2	11	7
token4	6	14	1	10	0	15	8	3

根据tokenIndex获取topk个idx

4

topkIdx - token3

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token3	15	0	4	13	9	2	11	7

根据tokenIndex获取
token的topk个weight

5

topk_weight - token3

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token3	0.85	0.08	0.03	0.02	0.01	0.005	0.003	0.002

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token0	0	0	0	0	0	0	0	0
token1	1	1	1	1	0	0	0	1
token2	1	1	2	0	2	0	0	0
token3	3	3	0	1	2	2	1	2
token4	1	1	1	1	4	4	1	1

根据tokenIndex拷贝token的小偏移

6

send_token_idx - token0

	topk0	topk1	topk2	topk3	topk4	topk5	topk6	topk7
token3	3	3	0	1	2	2	1	2

7

遍历topk个expert，准备如下内容，
以token3的第5个专家为例

expert_id = 2, src_rank = 2 / 4 = 0

8

取对应位置的weight

expert weight = 0.005

9

取对应位置的小偏移

小偏移 = 2, dispatch时写的小偏移，也是combine时读的小偏移，大偏移同理

10

取所属的大偏移

11

根据src_rank、大小偏移拷贝到本地

DataCopy(local_tmp, src[大偏移 + 小偏移])

local_float = Cast(local_tmp, CAST_NONE)

local_float_mul = local_float * expert_weight

local_sum += local_float_nul

topk次累加后，local_out = Cast(local_sum, CAST_RINT)

putOffset取：expert_id * rank_num + local_rank = 2 * 4 + 0 槽位 = 43，即读的大偏移

全量putOffset，即大偏移

	expert 0	expert 1	expert 2	expert 3
rank0	0 5 14 21	24 26 38 42	43 46 49 59	63 65 80 82
rank1	expert 4	expert 5	expert 6	expert 7
	0 1 1 9	14 16 24 25	27 29 40 49	49 52 59 62
rank2	expert 8	expert 9	expert 10	expert 11
	0 2 7 13	14 17 23 23	25 27 41 46	50 52 62 66
rank3	expert 12	expert 13	expert 14	expert 15
	0 2 6 8	13 15 28 36	37 39 44 45	45 50 52 58

输出：xOut，即token和weight加权求和结果

xOut[tokenIndex] = local_out

local2gm